МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-67

Выполнил
В. М. Голуб,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Вариант 1

Классификация ирисов
• Iris
• **Задача**: Определить вид ириса (3 класса).
• **Архитектура**:
  - входной слой;
  - один скрытый слой с 10 нейронами (ReLU);
  - выходной слой с 3 нейронами (Softmax).

• Эксперимент: попробуйте добавить второй скрытый слой с 5 нейронами и сравните точность.
Выполнение:

Код программы

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

torch.manual_seed(42)
np.random.seed(42)

# Загружаем данные из файла, который лежит в папке
data = pd.read_csv("iris.csv")

features = [
    "sepal.length",
    "sepal.width",
    "petal.length",
    "petal.width"
]

X = data[features].values

encoder = LabelEncoder()
y = encoder.fit_transform(data["variety"])

print("Количество записей:", len(data))
print("Количество признаков:", X.shape[1])
print("Классы:", encoder.classes_)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)


class MLP(nn.Module):
    # ВОТ ЗДЕСЬ БЫЛА ОШИБКА! Должно быть __init__ (два подчеркивания с каждой стороны)
    def __init__(self, input_size, second_layer=False):
        super().__init__()  # И здесь тоже __init__

        if second_layer:
            self.layers = nn.Sequential(
                nn.Linear(input_size, 10),
                nn.ReLU(),
                nn.Linear(10, 5),
                nn.ReLU(),
                nn.Linear(5, 3)
            )
        else:
            self.layers = nn.Sequential(
                nn.Linear(input_size, 10),
                nn.ReLU(),
                nn.Linear(10, 3)
            )

    def forward(self, x):
        return self.layers(x)


def train_model(second_layer, name):
    torch.manual_seed(42)

    model = MLP(X_train.shape[1], second_layer)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.01)

    for epoch in range(1, 201):
        model.train()

        output = model(X_train)
        loss = criterion(output, y_train)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if epoch % 50 == 0:
```

```
            print(f"{name} | Эпоха {epoch}, Loss: {loss.item():.4f}")

    model.eval()

    with torch.no_grad():
        output = model(X_test)
        probabilities = torch.softmax(output, dim=1)
        predictions = torch.argmax(probabilities, dim=1).numpy()

    actual = y_test.numpy()

    accuracy = accuracy_score(actual, predictions)

    print(f"\n{name}")
    print(f"Accuracy: {accuracy:.4f}")

    print("Confusion Matrix:")
    print(confusion_matrix(actual, predictions))

    print("Classification Report:")
    print(classification_report(
        actual,
        predictions,
        target_names=encoder.classes_,
        digits=4
    ))

    return accuracy


print("\nОБУЧЕНИЕ ПЕРВОЙ МОДЕЛИ")
accuracy_1 = train_model(False, "MLP с одним скрытым слоем")

print("\nОБУЧЕНИЕ ВТОРОЙ МОДЕЛИ")
accuracy_2 = train_model(True, "MLP с двумя скрытыми слоями")

print("\nСРАВНЕНИЕ РЕЗУЛЬТАТОВ")
print(f"Один скрытый слой: {accuracy_1:.4f}")
print(f"Два скрытых слоя:  {accuracy_2:.4f}")

if accuracy_2 > accuracy_1:
    print("Добавление второго слоя улучшило точность.")
elif accuracy_2 < accuracy_1:
    print("Добавление второго слоя снизило точность.")
else:
    print("Добавление второго слоя не изменило точность.")
```

Результат:

Количество записей: 150
Количество признаков: 4
Классы: ['Setosa' 'Versicolor' 'Virginica']

ОБУЧЕНИЕ ПЕРВОЙ МОДЕЛИ

MLP с одним скрытым слоем | Эпоха 50, Loss: 0.2056
MLP с одним скрытым слоем | Эпоха 100, Loss: 0.0714
MLP с одним скрытым слоем | Эпоха 150, Loss: 0.0497
MLP с одним скрытым слоем | Эпоха 200, Loss: 0.0413

MLP с одним скрытым слоем
Accuracy: 0.9667
Confusion Matrix:
[[10 0 0]
 [0 9 1]
 [0 0 10]]
Classification Report:

	precision	recall	f1-score	support
Setosa	1.0000	1.0000	1.0000	10
Versicolor	1.0000	0.9000	0.9474	10
Virginica	0.9091	1.0000	0.9524	10
accuracy			0.9667	30
macro avg	0.9697	0.9667	0.9666	30
weighted avg	0.9697	0.9667	0.9666	30

ОБУЧЕНИЕ ВТОРОЙ МОДЕЛИ
MLP с двумя скрытыми слоями | Эпоха 50, Loss: 0.5021
MLP с двумя скрытыми слоями | Эпоха 100, Loss: 0.1436
MLP с двумя скрытыми слоями | Эпоха 150, Loss: 0.0851
MLP с двумя скрытыми слоями | Эпоха 200, Loss: 0.0639

MLP с двумя скрытыми слоями
Accuracy: 0.9667
Confusion Matrix:
[[10 0 0]
 [0 9 1]
 [0 0 10]]
Classification Report:

	precision	recall	f1-score	support
Setosa	1.0000	1.0000	1.0000	10
Versicolor	1.0000	0.9000	0.9474	10
Virginica	0.9091	1.0000	0.9524	10
accuracy			0.9667	30
macro avg	0.9697	0.9667	0.9666	30
weighted avg	0.9697	0.9667	0.9666	30

СРАВНЕНИЕ РЕЗУЛЬТАТОВ
Один скрытый слой: 0.9667
Два скрытых слоя: 0.9667
Добавление второго слоя не изменило точность.

Вывод: В ходе лабораторной работы построена и обучена нейронная сеть (MLP) для классификации ирисов. В рамках эксперимента были сравнены две архитектуры: с одним скрытым слоем (10 нейронов) и с двумя (10 и 5 нейронов). Обе модели показали одинаковую точность на тестовой выборке — **96.67%** (допустив всего 1 ошибку).